

Abdullah Haroon

haroon

Path 3

<https://github.com/ECEDataScience/miniproject-s24-AbdullahHaroon1604>

<https://github.com/ECEDataScience/miniproject-s24-AbdullahHaroon1604.git>

Dataset Description:

I am going to use digit dataset which is a built-in dataset in sklearn. Images of handwritten digits are gathered in the digit dataset, which is frequently used to train and evaluate machine learning algorithms, particularly those that deal with pattern recognition and picture categorization. There are ten classes (0–9) in this dataset, and each class consists of eight by eight pixel images with handwritten numbers 0–9. There are 10,000 testing photos and 60,000 training images in the dataset. Each image has its matching digit tagged on it.

This dataset is very useful for understanding machine learning classification techniques and assessing how well various machine learning algorithms work, particularly when it comes to picture classification algorithms.

Method:

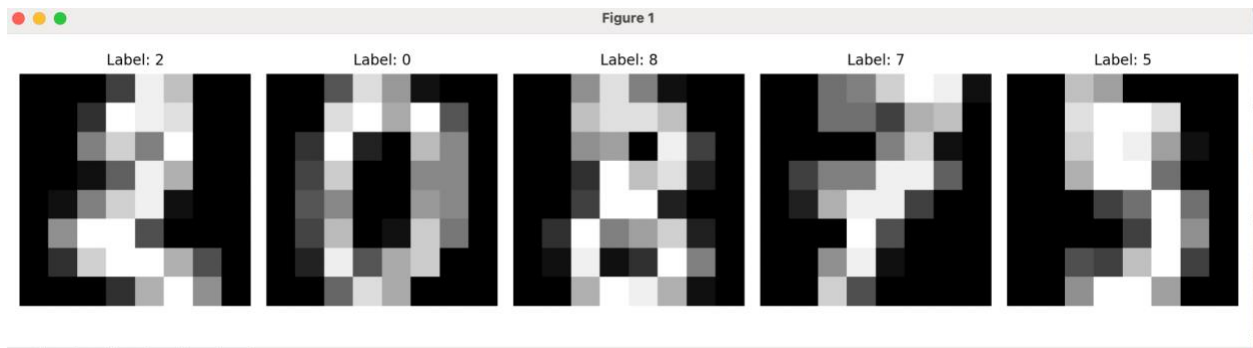
The analysis I am going to perform is evaluate the performances of machine learning algorithm GaussianNB, KNN (K-Nearest Neighbors), and MLP (Multilayer Perceptron) on the digit dataset, among these three algorithms which algorithm performs better in recognizing the image dataset, how resilient it is when we add a noise to the dataset and what could be the affect on the performance when we denoise the dataset.

GaussianNB, KNN, and MLP Classifier are all popular machine learning algorithms used for classification tasks. They all have their own strengths and weaknesses and that's what I am to evaluate their performances on the digits dataset in my project.

I am going to analyze different machine learning algorithms on this dataset to compare their performances to classify this data. We can use different parameters to compare the performances however here we are calculating the correctness of each model to come up with the best model for this case. Correctness we can calculate by counting how many predicted values are correct from the original values.

Result & Analysis:

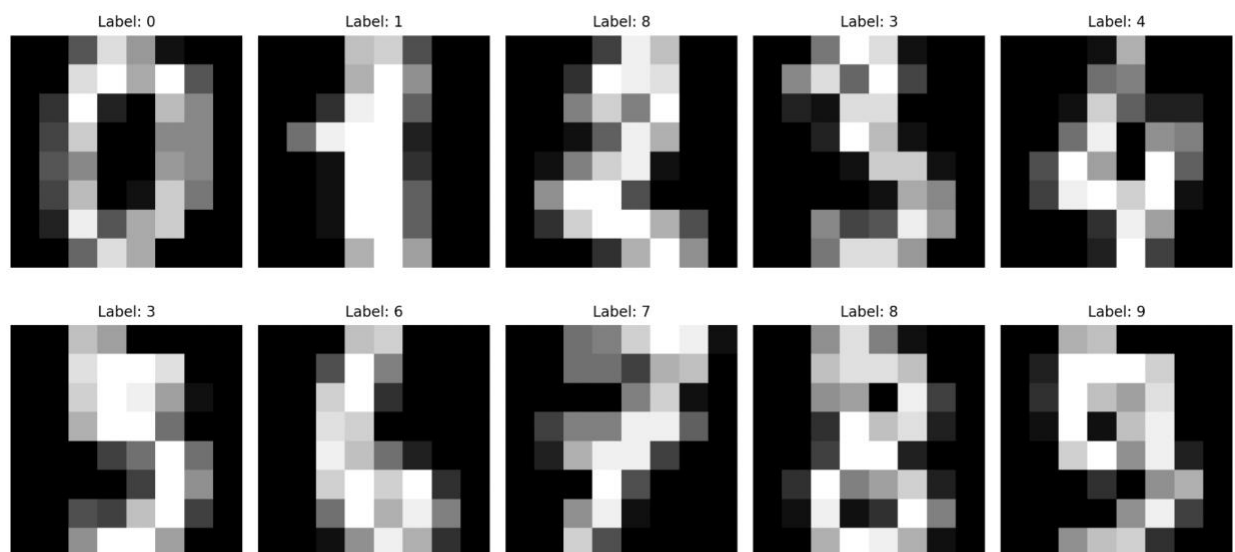
After importing the dataset, the first ask is to print the numbers for classes [2, 0, 8, 7, 5], here is the plot:



From the above plot we can see the images for corresponding classes (digit) and compare them with their labels.

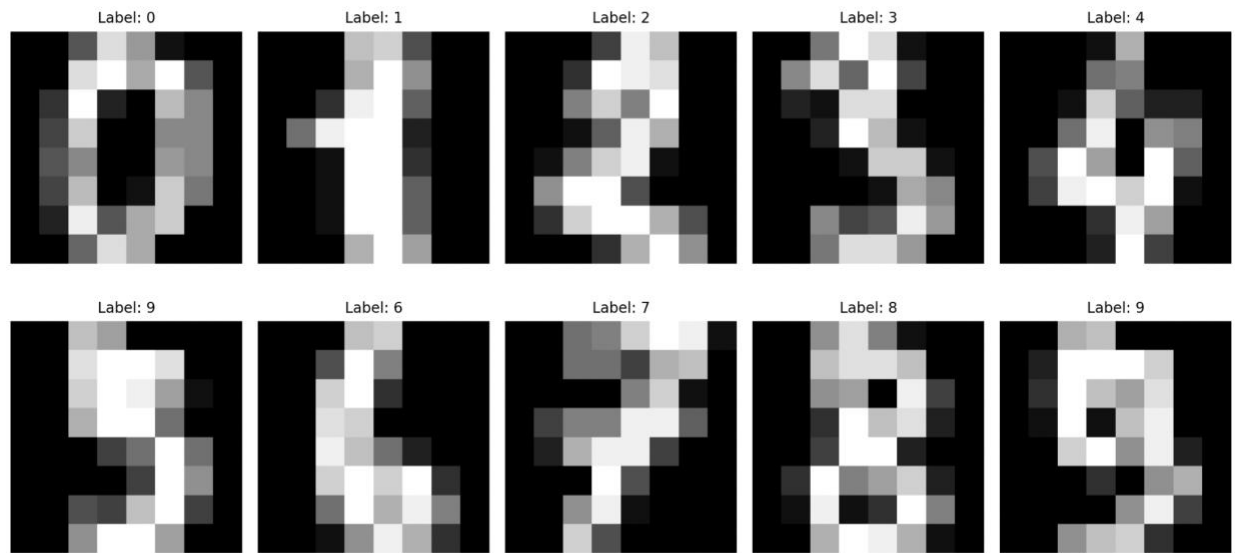
Gaussian Naive Bayes (GaussianNB):

After training this model and getting the predict values for test data, the overall accuracy we get for this model is 80%. GaussianNB is a good model to handle high dimensional data however it is not that efficient for pixel values in an image, we can see from the below plot that classes (digits) 2, 5 have wrong labels and were not correctly recognized correctly, and accuracy is 80%. We expect that this model will show us 80% accuracy, which would generally be 8 out of the 10 classes shown.



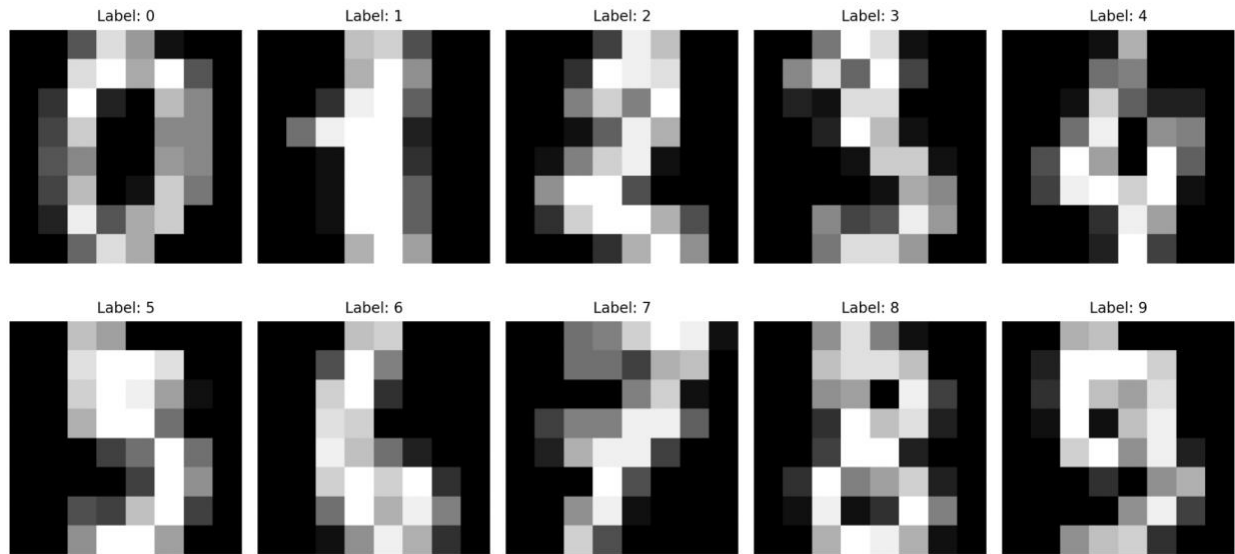
K-Nearest Neighbors (KNN):

KNN can perform well when there are clear clusters in the feature space, which might occur if similar digits are close together in the data. We see the accuracy of 95% and only one class (digit) 5 labeled wrongly with 9. We expect that this model will show us 95% accuracy, which would generally be 9 or 10 out of the 10 classes shown.



MLP Classifier:

MLPs can automatically learn hierarchical representations of features, which can be useful for capturing complex patterns in image data like handwritten digits. We see the accuracy of 91% for this model however all the classes (digits) were recognized correctly. We expect that this model will show us 91% accuracy, which would generally be 9 out of the 10 classes shown.



Analysis of Three Models:

As we have discussed the strengths of each model and analyzed their overall accuracy and plotted for each of the model, MLP Classifier and KNN are best models in analyzing digital images where the similar pixels are close to each other. Even though the accuracy of KNN is 95% however MLP Classifier with accuracy of 91% was able to label all the classes correctly. We expect a accuracy range of generally 80% or above on any given model.

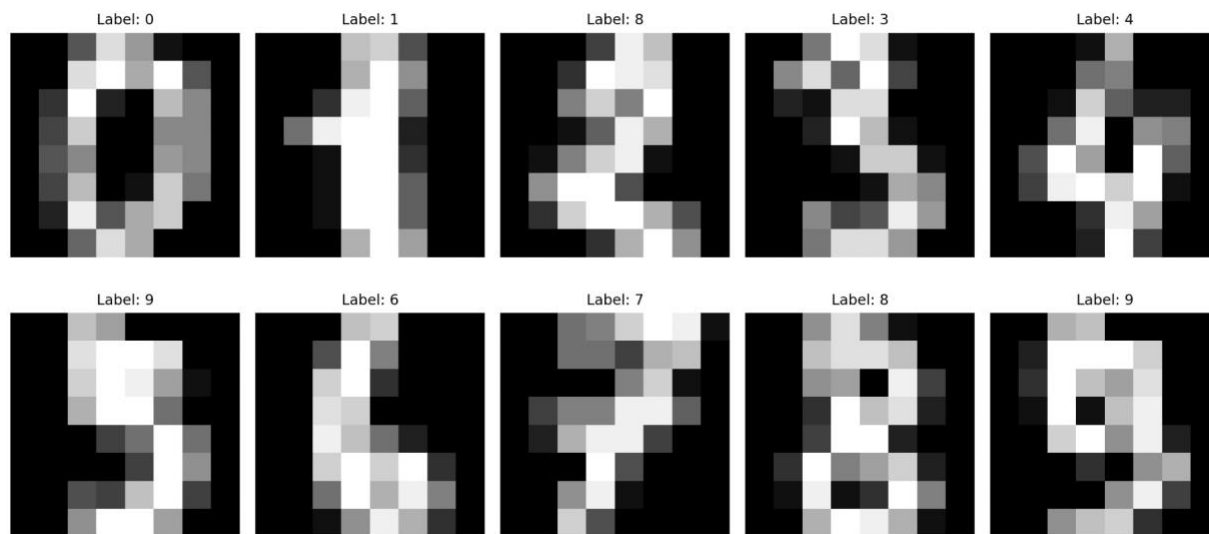
Data Poisoning:

We are generating Gaussian noise with a standard deviation of 10 for each element in the training data. Then we add this noise to the original training data `x_train` to create the "poisoned" training data `x_train_poison`. We are going to analyze three models using poison data to see which model's performance changed significantly and which model is shows strongest robustness against the poisoning. We except that since the poisoned data is harder to read, the general accuracy of each of the models will be lower.

Model: **GaussianNB**

Accuracy: 81%

Plot:



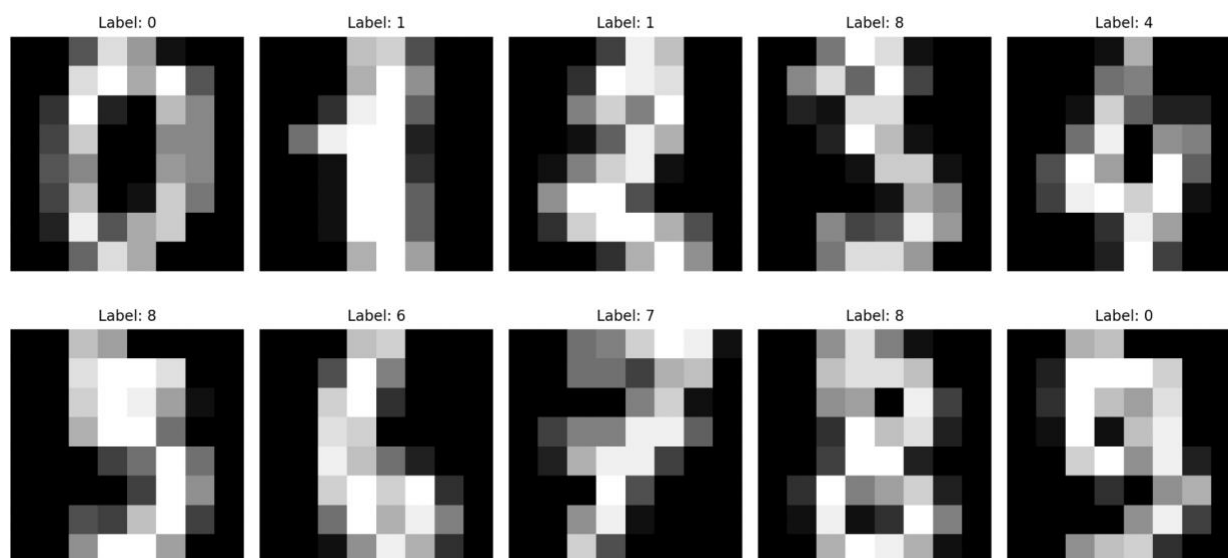
Analysis:

This model seems to show strongest robustness against poisoning, the overall accuracy was 80% before poisoning the data while it is 81% after the poison.

Model: **KNN**

Accuracy: 61%

Plot:



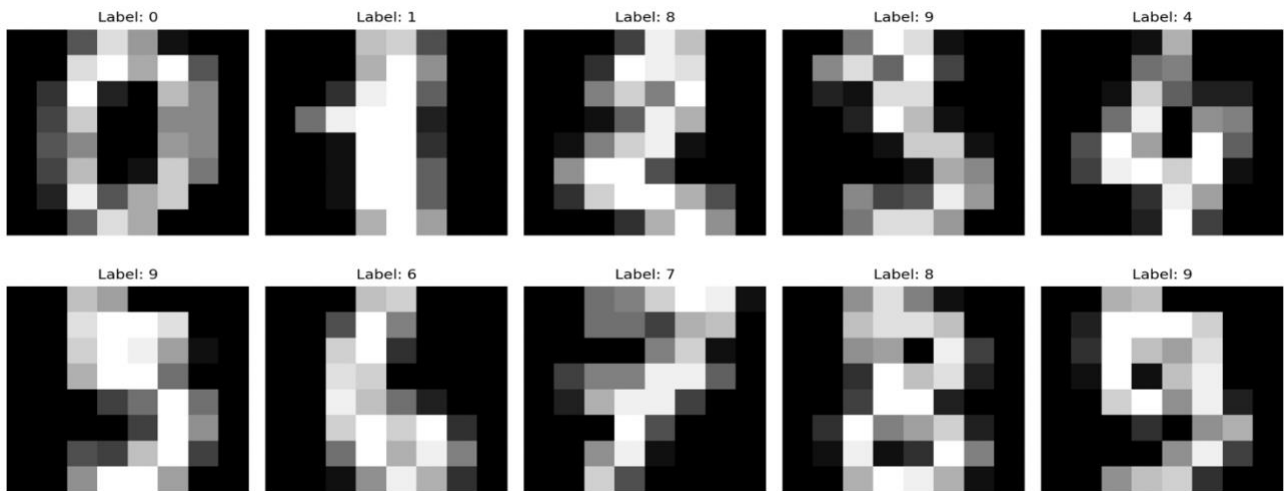
Analysis:

This model impacted the most by poisoning the data, the overall accuracy dropped from 95% to 61%.

Model: **GaussianNB**

Accuracy: 79%

Plot:



Analysis:

This model also got impacted by poisoning but not as much as KNN model, this model's accuracy dropped from 91% to 79%.

Denoise the Poison Data:

We like to perform analysis of these three models after denoising the poison training data.

We are using kernelPCA (Kernel Principal Component Analysis) method from sklearn library to denoise the data. KernelPCA is a nonlinear dimensionality reduction technique that maps the input data into a higher-dimensional feature space.

We are using kernel 'rbf' or 'gaussian': Radial Basis Function (RBF), the RBF kernel is popular because of its versatility, flexibility, and effectiveness in capturing nonlinear relationships in the data. However, we must carefully tune the hyperparameters, especially gamma, to achieve optimal performance.

The gamma parameter controls the kernel's width. Tuning the gamma parameter allows you to adjust the smoothness of the decision boundaries. Higher values of gamma lead to more complex decision boundaries and can potentially overfit the data, while lower values may underfit. This tuneability provides flexibility in adapting the kernel to different datasets and problem settings.

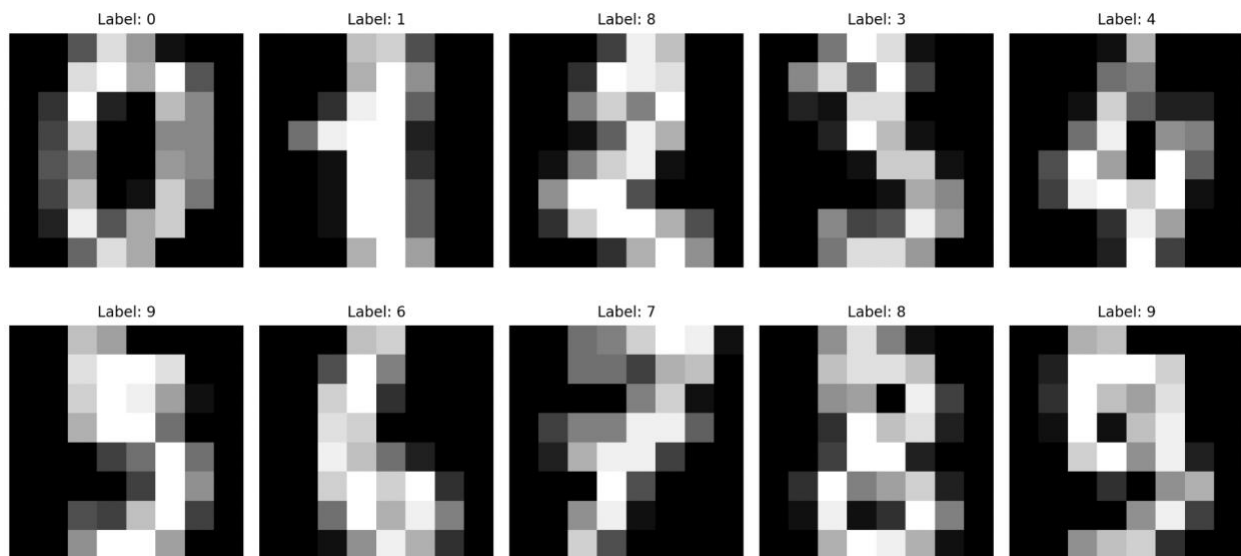
I have tried different values of gamma and came up with the value of 0.00001 (1e-5) where I get the best results for MLP Classifier model. We expect that since the denoised data should be more clear and easier to read, the accuracy of the models should be generally higher.

Here are the results of each model after denoising the poison data.

Model: **GaussianNB**

Accuracy: 82%

Plot:



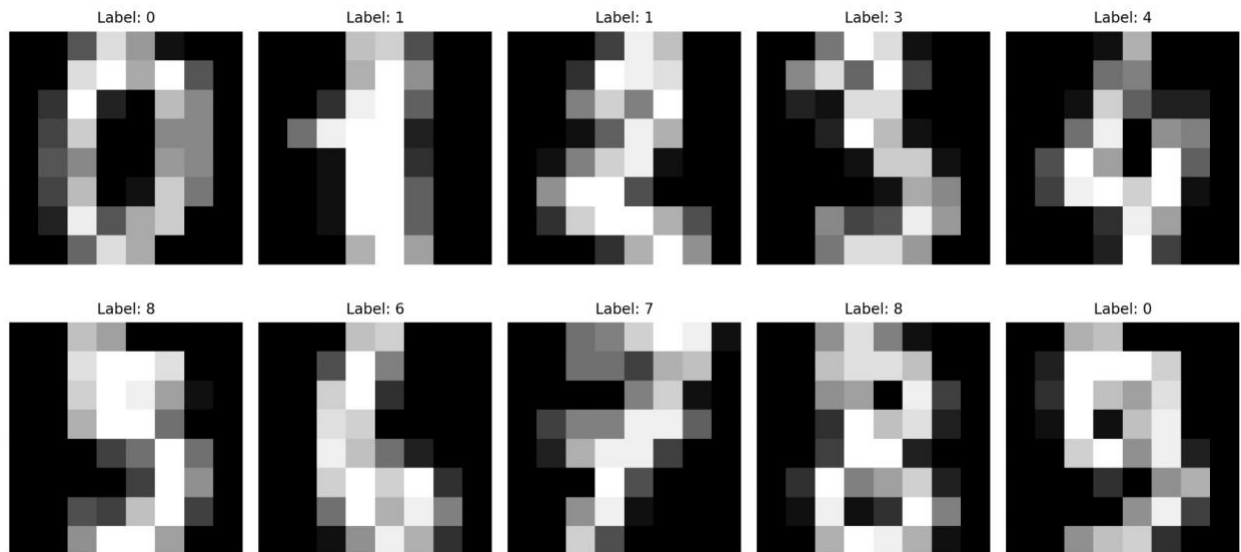
Analysis:

This model got very little improvement of 1% in overall accuracy, this model was not impacted by the poison data as well. So, it stayed almost the same in every case.

Model: **KNN**

Accuracy: 65%

Plot:

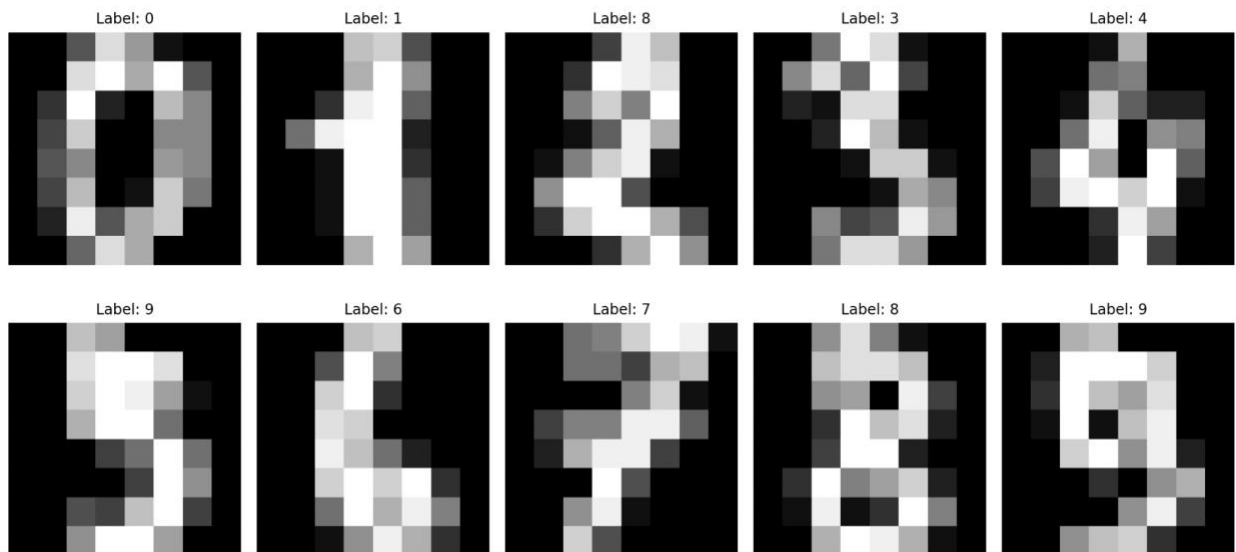


Analysis: This model got little improved after denoising the poisoning data, the overall accuracy went up from 61% to 65%, not a huge improvement though.

Model: **MLP Classifier**

Accuracy: 87%

Plot:



Analysis: We see some improvements in this model and jumped from 79% to 87% correctness, this value is very close to the original value of correctness prior to poisoning the data of 91%.